Practical 6

Aim: Implement a Semantic Analyzer: Define semantic rules for declaration statements to update the data types of symbols in the symbol table. Perform semantic analysis to detect and report the use of undeclared variables and redeclaration of variables.

Prac6.l:

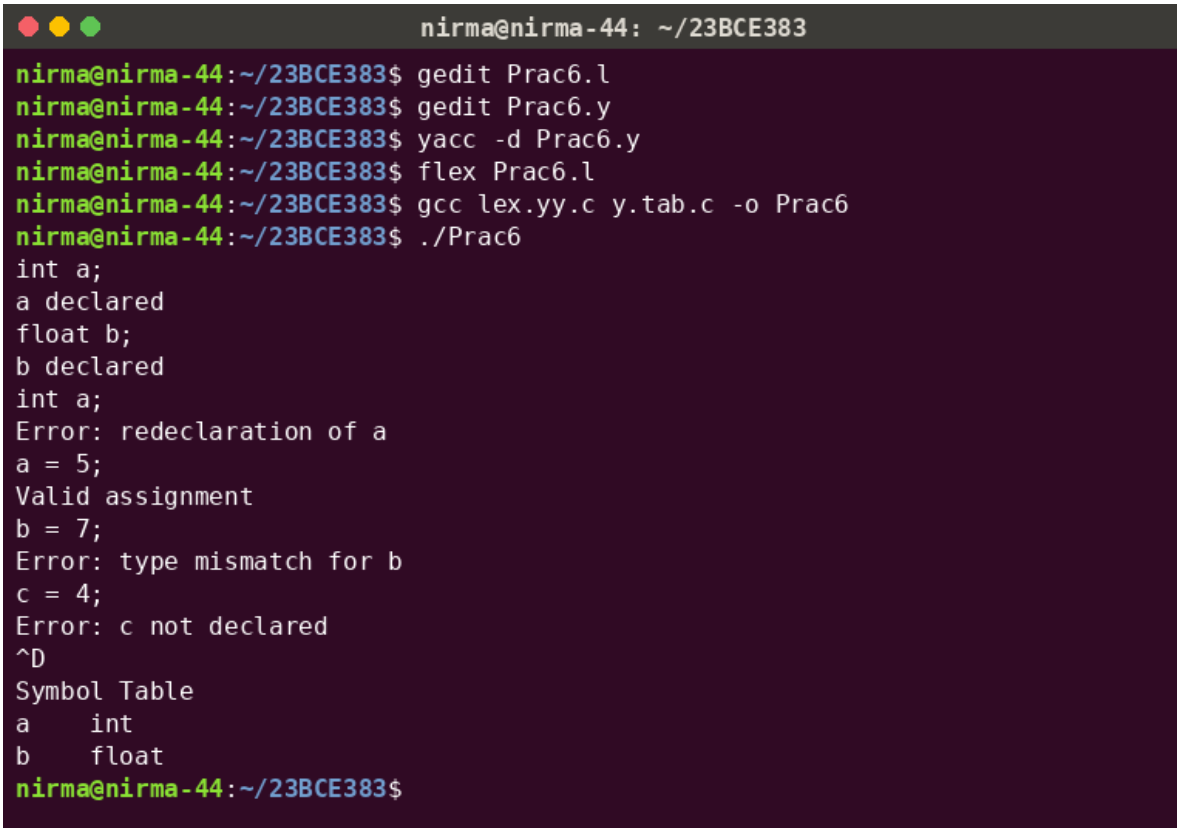
```
%{
#include "y.tab.h"
#include<string.h>
extern int yylval;
extern char id[20];
%}

%%
int             {yylval=0; return TYPE;}
float           {yylval=1; return TYPE;}
[0-9]+          {yylval=0; return NUMBER;}
[0-9]+\.[0-9]+  {yylval=1; return NUMBER;}
[a-z]+          {strcpy(id,yytext); return ID;}
[ \t]           ;
\n              {return '\n';}
.               {return yytext[0];}
%%

int yywrap()
{
return 1;
}
```

Prac6.y:

```
%{
#include<stdio.h>
#include<string.h>
int yylex();
void yyerror(char *s);
char id[20], name[20][20];
int type[20], n=0, i;
int find();
%}

%token TYPE NUMBER ID
%%
input: input stmt
     | stmt
     ;
stmt: TYPE ID ';' '\n' {
        if(find()!=-1) printf("Error: redeclaration of %s\n",id);
        else {strcpy(name[n],id); type[n++]=$1; printf("%s declared\n",id);}
      }
    | ID '=' NUMBER ';' '\n' {
        i=find();
        if(i==-1) printf("Error: %s not declared\n",id);
        else if(type[i]!=$3) printf("Error: type mismatch for %s\n",id);
        else printf("Valid assignment\n");
      }
    ;
%%
```

```
int find()
{
for(i=0;i<n;i++)
  if(strcmp(name[i],id)==0) return i;
return -1;
}

void yyerror(char *s)
{
printf("Syntax error\n");
}

int main()
{
yyparse();
printf("\nSymbol Table\n");
for(i=0;i<n;i++)
  printf("%s\t%s\n",name[i],type[i]==0?"int":"float");
return 0;
}
```

Output:



Conclusion:

In this practical declarations were stored in the symbol table along with their data types. Undeclared variables, redeclaration and type mismatch errors were detected and reported.